

Mesin Bola

Madina telah menciptakan mesin bola untuk menghibur para peserta IOI. Struktur internal dari mesin tersebut adalah sebagai berikut:

- Mesin tersebut terdiri dari N **verteks**, yang dinomori dari 0 hingga $N - 1$. Verteks $N - 1$ disebut **akar**.
- Untuk setiap verteks u dengan $0 \leq u < N - 1$, **parent** dari verteks u adalah verteks $P[u]$ dengan $P[u] > u$, dan verteks u adalah **anak** dari $P[u]$. Akar tidak memiliki **parent**. Verteks yang tidak memiliki anak disebut sebagai **daun**.

Anda **tidak** tahu N ataupun **parent** $P[u]$ dari verteks u mana pun. Sebagai gantinya, Madina memberi tahu Anda M : banyaknya daun, dan bahwa daun-daun tersebut adalah verteks bernomor $0, 1, \dots, M - 1$. Tugas Anda adalah menentukan struktur internal mesin dengan mengoperasikannya.

Setiap verteks pada mesin dapat menampung paling banyak satu **bola**, dan setiap bola memiliki **nilai** yang merupakan bilangan bulat non-negatif. Kita sebut suatu verteks bernilai x jika verteks tersebut menampung bola bernilai x . Suatu verteks dikatakan **terisi** jika berisi bola; jika tidak, verteks tersebut **kosong**. Awalnya, setiap verteks kosong.

Anda dapat melakukan dua jenis operasi:

- **insert**(U, X): mencoba memasukkan bola baru dengan nilai X pada daun U .
 - Jika daun U terisi, maka operasi ini mengembalikan `false` tanpa memasukkan bola.
 - Jika daun U kosong, maka bola diletakkan pada verteks U . Kemudian, bola itu secara berulang-ulang berpindah ke **parent** dari verteks yang ditempati bola saat ini selama **parent** tersebut kosong. Perpindahan ini berhenti ketika bola mencapai akar, atau mencapai sebuah verteks yang **parent**-nya terisi. Operasi ini akan mengembalikan nilai `true`.
- **collect**(): Jika akar kosong (yang berarti mesin juga kosong), `collect` mengembalikan *array* kosong. Jika tidak, prosedur rekursif `traverse` yang dijelaskan di bawah ini mengumpulkan nilai semua bola yang saat ini ada pada mesin ke dalam *array* S . Awalnya, *array* S kosong dan `traverse(N - 1)` dipanggil.

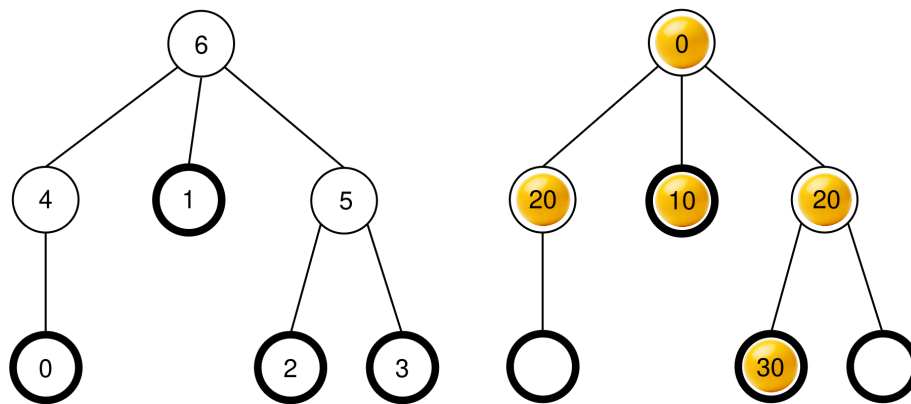
```

traverse(u) :
    tambahkan nilai dari bola pada verteks u ke akhir S
    misalkan c[u] adalah daftar semua anak dari u yang terisi
    urutkan c[u] secara tidak menurun berdasarkan nilai-nilai bola
    pada verteks-verteks tersebut (jika ada beberapa verteks
    dengan nilai yang sama, urutan verteks-verteks itu bebas)
    untuk setiap v dalam c[u]:
        traverse(v)

```

Setelah prosedur ini selesai, **semua bola dibuang** dari mesin, dan `collect` mengembalikan S .

Sebagai contoh, perhatikan mesin dengan $N = 7$ verteks dan *array parent* $P = [4, 6, 5, 5, 6, 6]$. Gambar sebelah kiri di bawah ini menunjukkan sebuah mesin kosong dengan semua nomor verteksnya, sementara gambar sebelah kanan menunjukkan suatu konfigurasi bola yang mungkin setelah serangkaian operasi `insert` (rangkaiannya ini diberikan pada bagian Contoh).



Ketika `collect()` dipanggil, akar (yaitu verteks 6) sedang terisi, sehingga nilai awal S adalah $S = []$ dan `traverse(6)` dipanggil.

traverse(6): verteks 6 memiliki nilai 0; menembarkannya ke S menghasilkan $S = [0]$. Anak-anak dari verteks 6 adalah 4, 1, dan 5, yang semuanya terisi. Nilai masing-masing adalah 20, 10, dan 20. Pengurutan tidak menurun menghasilkan $c[6] = [1, 5, 4]$ (perhatikan bahwa $c[6] = [1, 4, 5]$ juga valid, karena verteks 4 dan 5 memiliki nilai yang sama). Prosedur ini kemudian memanggil `traverse` pada setiap verteks di $c[6]$ dalam urutan tersebut.

- **traverse(1):** verteks 1 memiliki nilai 10; menembarkannya ke S menghasilkan $S = [0, 10]$. Verteks 1 tidak memiliki anak yang terisi, sehingga pemanggilan fungsi ini berakhir.
- **traverse(5):** verteks 5 memiliki nilai 20; menembarkannya ke S menghasilkan $S = [0, 10, 20]$. Verteks 5 memiliki satu anak terisi, yakni verteks 2, sehingga $c[5] = [2]$ dan prosedur ini memanggil `traverse(2)`.
 - **traverse(2):** verteks 2 memiliki nilai 30; menembarkannya ke S menghasilkan $S = [0, 10, 20, 30]$. Verteks 2 tidak punya anak yang terisi, sehingga pemanggilan

fungsi ini berakhir.

- Eksekusi kembali ke `traverse(5)` yang telah memproses semua anak pada `c[5]`, sehingga pemanggilan ini berakhir.
- **`traverse(4)`**: verteks 4 memiliki nilai 20; menambahkannya ke S menghasilkan $S = [0, 10, 20, 30, 20]$. Verteks 4 tidak memiliki anak yang terisi, sehingga pemanggilan ini berakhir.

Semua pemanggilan telah berakhir dan tidak ada verteks lagi yang perlu diproses. Terakhir, semua bola dibuang dari mesin dan `collect` mengembalikan $S = [0, 10, 20, 30, 20]$.

Tugas Anda adalah menentukan banyaknya verteks, N , dan melaporkan *array parent* $R = [R[0], R[1], \dots, R[N-2]]$ yang menggambarkan struktur dari mesin, tetapi dengan penomoran yang mungkin saja berbeda untuk verteks yang bukan daun atau akar. Secara formal, sebuah *array* R yang dilaporkan pada jawaban dianggap benar jika terdapat sebuah penomoran unik $L[u]$ dengan $0 \leq L[u] < N$ pada setiap verteks u dari mesin, sedemikian rupa sehingga:

- $L[u] = u$ untuk semua $0 \leq u < M$ dan $u = N - 1$, dan
- $L[P[u]] = R[L[u]]$ untuk semua $0 \leq u < N - 1$.

Tidak diharuskan bahwa $R[u] > u$. Mesin tersebut **harus kosong** ketika Anda melaporkan jawaban Anda.

Misalkan K adalah banyaknya operasi `collect` yang dilakukan, dan B adalah nilai maksimum semua bola dari semua pemanggilan `insert` yang dilakukan. Misalkan $C = K + B$. Maka, C **tidak boleh melebihi** 1000. Nilai yang Anda dapatkan pada beberapa subsoal bergantung pada nilai C .

Detail Implementasi

Anda harus mengimplementasikan prosedur berikut.

```
std::vector<int> find_structure(int M)
```

- M : banyaknya daun pada mesin.
- Prosedur ini dipanggil tepat sekali untuk setiap kasus uji.
- Prosedur ini harus mengembalikan *array* $R = [R[0], R[1], \dots, R[N-2]]$ dengan panjang $N - 1$, yakni suatu *array parent* yang benar.

Untuk berinteraksi dengan mesin, prosedur Anda dapat memanggil dua prosedur di bawah ini.

Prosedur pertama adalah:

```
bool insert(int U, int X)
```

- U : nomor daun tempat bola akan dimasukkan. Harus terpenuhi bahwa $0 \leq U < M$.
- X : nilai bilangan bulat pada bola. Harus terpenuhi bahwa $0 \leq X \leq 1000$.
- Prosedur ini mengembalikan `true` jika bola berhasil dimasukkan, atau `false` jika daun U sudah terisi.
- Prosedur ini dapat dipanggil paling banyak 500 000 kali.

Prosedur kedua adalah:

```
std::vector<int> collect()
```

- Mengumpulkan nilai-nilai semua bola yang telah dimasukkan, mengosongkan mesin, lalu mengembalikan *array* S .

Jika pada suatu waktu saat program Anda dijalankan, nilai C melebihi 1000, jawaban Anda akan menerima hasil `Output isn't correct: Too many resources used`.

Struktur mesin sudah ditentukan sebelum `find_structure` dipanggil. *Grader* bersifat **deterministik**: jika Anda menjalankannya dua kali dan keduanya melakukan rangkaian operasi yang sama, pemanggilan `collect` akan menghasilkan *array* yang sama.

Batasan

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subsoal

Subsoal	Nilai	Batasan tambahan
1	5	Akar memiliki tepat M anak.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Tidak ada batasan tambahan.

Pada subsoal 3 dan 4, nilai Anda bergantung pada nilai C sebagai berikut.

Subsoal 3 (25 poin)

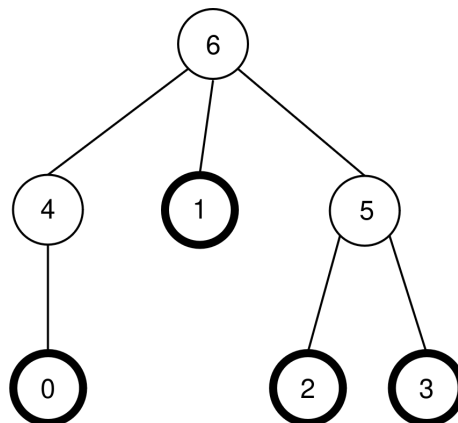
Batasan	Nilai
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subsoal 4 (60 poin)

Batasan	Nilai
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Contoh

Perhatikan mesin yang sama dengan yang diberikan pada deskripsi soal, dengan $N = 7$, $M = 4$, dan $P = [4, 6, 5, 5, 6, 6]$. Mesin ini ditunjukkan pada gambar berikut:



Grader melakukan pemanggilan sebagai berikut:

```
find_structure(4)
```

Prosedur tersebut dapat melakukan serangkaian pemanggilan sebagai berikut:

1. **insert(0, 0)**: daun 0 sedang kosong, sehingga bola dengan nilai 0 diletakkan pada verteks 0. Verteks $P[0] = 4$ sedang kosong, sehingga bola berpindah ke verteks 4. Verteks

$P[4] = 6$ juga kosong, sehingga bola berpindah ke verteks 6. Karena verteks 6 adalah akar, perpindahan ini berhenti. Pemanggilan ini mengembalikan nilai `true`.

2. `insert(3, 20)`: daun 3 sedang kosong, sehingga bola dengan nilai 20 diletakkan pada daun 3. Verteks $P[3] = 5$ juga sedang kosong, sehingga bola berpindah ke verteks 5. Karena $P[5] = 6$ terisi, maka perpindahan ini berhenti. Pemanggilan ini mengembalikan nilai `true`.

3. `insert(1, 10)`: daun 1 sedang kosong, sehingga bola dengan nilai 10 diletakkan pada daun 1. Karena $P[1] = 6$ terisi, bola ini tetap pada verteks 1. Pemanggilan ini mengembalikan nilai `true`.

4. `insert(2, 30)`: daun 2 sedang kosong, sehingga bola dengan nilai 30 diletakkan pada daun 2. Karena $P[2] = 5$ terisi, bola ini tetap pada verteks 2. Pemanggilan ini mengembalikan nilai `true`.

5. `insert(1, 25)`: daun 1 terisi, sehingga bola ini tidak dapat diletakkan pada daun 1. Pemanggilan ini mengembalikan nilai `false`.

6. `insert(0, 20)`: daun 0 sedang kosong, sehingga bola dengan nilai 20 diletakkan pada daun 0. Verteks $P[0] = 4$ sedang kosong juga, sehingga bola berpindah ke verteks 4. Karena $P[4] = 6$ sedang terisi, perpindahan ini berhenti. Pemanggilan ini mengembalikan nilai `true`.

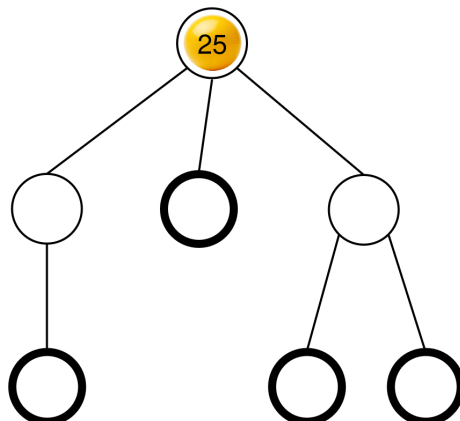
Konfigurasi bola yang dihasilkan telah ditunjukkan pada deskripsi soal.

Selanjutnya, prosedur ini dapat memanggil:

```
collect()
```

Eksekusi dari pemanggilan ini sudah dijelaskan pada deskripsi soal, dan pemanggilan ini menghasilkan $S = [0, 10, 20, 30, 20]$. Setelahnya, mesin ini kembali kosong.

Selanjutnya, prosedur ini dapat memanggil `insert(2, 25)`. Daun 2 sedang kosong, sehingga bola dengan nilai 25 diletakkan pada daun 0. Bola ini kemudian berpindah ke verteks 5, lalu ke verteks 6, tempat bola tersebut berhenti. Pemanggilan ini mengembalikan nilai `true`. Konfigurasi bola yang dihasilkan ditunjukkan pada gambar berikut.



Terakhir, prosedur dapat memanggil `collect()` yang mengembalikan $S = [25]$ dan mengosongkan mesin.

Ada dua kemungkinan *array parent* yang benar yang dapat dikembalikan oleh `find_structure`:
 $R = P = [4, 6, 5, 5, 6, 6]$ (yang sesuai dengan penomoran $L = [0, 1, 2, 3, 4, 5, 6]$), dan
 $R = [5, 6, 4, 4, 6, 6]$ (yang sesuai dengan penomoran $L = [0, 1, 2, 3, 5, 4, 6]$). Pada contoh ini, $K = 2$ dan $B = 30$, sehingga $C = 32$.

Contoh Grader

Format masukan:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Format keluaran:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Di sini, Q adalah panjang dari *array* R yang dikembalikan oleh `find_structure`.